Automatización de Tareas en SGBD

Sistemas de Gestión de Bases de Datos

Optimización de procesos, mejora de eficiencia operativa y reducción de errores humanos humanos

Fecha de informe: 2025-10-19

MariaDB & MySQL

Fundamentos y Ventajas de la Automatización

¿Qué es la automatización en SGBD?

Ejecución sistemática y repetitiva de tareas sin la intervención directa del usuario.

En un entorno corporativo real, los datos no residen en el ordenador del usuario: están en un servidor central de base de datos (MariaDB) al que se conectan múltiples aplicaciones cliente (interfaces web, paneles de administración, terminales o programas en Python/Java) a través de una red local o Internet.

Operación tradicional (Sin automatización interna): La aplicación cliente asume toda la responsabilidad. Para completar una tarea, el cliente debe redactar y enviar una secuencia de múltiples sentencias SQL sueltas a través del cable de red, esperar las respuestas intermedias del servidor, procesar los datos en la memoria del cliente y volver a enviar más órdenes.

Operación con automatización interna (En el SGBD): La lógica de gestión y las reglas operativas se programan y almacenan directamente en el servidor dentro de objetos como procedimientos, funciones o disparadores. La aplicación cliente solo envía una orden mínima con los parámetros necesarios (CALL ...), y el motor de base de datos ejecuta internamente todas las comprobaciones, cálculos y modificaciones directamente sobre los datos almacenados.

Principales Beneficios

Ahorro de Tiempo

Las tareas se ejecutan automáticamente en momentos predefinidos, liberando al personal de la necesidad de intervención manual constante.

Reducción de Costes

Al disminuir la necesidad de supervisión y mantenimiento manual, se optimizan los recursos operativos y se minimizan los gastos asociados a la intervención humana.

Minimización de Errores

La eliminación de la intervención humana en la ejecución de tareas repetitivas reduce drásticamente la probabilidad de errores, conduciendo a mayor consistencia y precisión.

Enfoques de Implementación

Scripts Externos al SGBD

Desarrollo de programas fuera del sistema gestor de bases de datos

Se ejecutan mediante herramientas del sistema operativo como Programador de Tareas (Cron en Linux, Task Scheduler en windows)

Se conectan al SGBD para ejecutar sentencias SQL necesarias

Ideal para tareas complejas que requieren lógica externa o integración con otros sistemas

Automatización Interna

Utiliza herramientas de programación del propio motor de base de datos

Encapsula la lógica de automatización directamente dentro del SGBD

Aprovecha capacidades nativas para gestión y ejecución de tareas

Preferible para tareas relacionadas directamente con los datos y la base de datos

Comparativa

- La automatización interna ofrece mayor eficiencia ya que opera directamente sobre los datos
- Los scripts externos son más versátiles para tareas complejas o que requieren integración multi-sistema

Ventajas de la Automatización Interna

La automatización integrada en el SGBD ofrece beneficios clave que no están disponibles con métodos externos:

Independencia del SO

Las tareas se ejecutan directamente en el motor de la base de datos, sin afectar su funcionamiento por migraciones entre plataformas.

- Funcionamiento consistente entre Linux y Windows

Gestión Centralizada

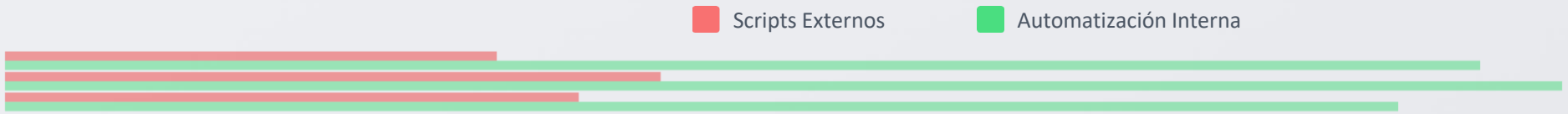
Las reglas de negocio y tareas se almacenan junto con los datos, facilitando su control mediante comandos SQL estándar.

- Auditoría y modificación simplificadas

Mayor Eficiencia

Operando directamente sobre los datos internos, el motor ejecuta tareas sin conexiones externas ni autenticaciones adicionales.

- Ejecución más rápida y menor consumo de recursos



Objetos Programables en MariaDB y MySQL

Procedimientos Almacenados

Conjuntos de sentencias SQL que se compilan y almacenan en el servidor. Encapsulan lógica de negocio compleja y tareas repetitivas.

Se ejecutan con una simple llamada

Funciones Definidas por el Usuario

Similares a los procedimientos almacenados, pero siempre devuelven un valor. Útiles para cálculos específicos y transformaciones de datos.

Se invocan directamente en sentencias SQL

Triggers (Disparadores)

Objetos programables que se ejecutan automáticamente en respuesta a eventos de manipulación de datos (DML) como INSERT, UPDATE y DELETE.

Mantienen integridad de datos y validaciones

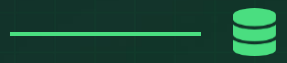
Eventos Programados

Permiten la ejecución de tareas SQL en momentos específicos o de forma recurrente. Esenciales para el mantenimiento periódico de la base de datos.

Similar a "cron jobs" del sistema operativo

Procedimientos Almacenados

Funcionalidad y Aplicación



Bloques de código SQL reutilizables que se guardan en el servidor y pueden ejecutarse cuando se necesite

```
CREATE PROCEDURE nombre_procedimiento (parámetros)
```

19 de octubre de 2025

¿Qué es un Procedimiento Almacenado?

Un procedimiento almacenado es un bloque de código SQL que se guarda directamente en el servidor de la base de datos.

Funciona como una "función" que encapsula lógica reutilizable, permitiendo su ejecución bajo demanda y facilitando la organización y eficiencia en la gestión de datos.



Características Principales:

Reutilizable

Se puede llamar y ejecutar varias veces desde diferentes aplicaciones y usuarios

Almacenamiento en Servidor

El código reside directamente en la base de datos, lo que facilita su gestión y seguridad

Seguridad

Permite controlar el acceso a los datos mediante permisos específicos y centraliza la lógica de negocio

Ventajas y Utilidades

Los procedimientos almacenados ofrecen múltiples beneficios que optimizan la gestión y el rendimiento de las bases de datos:

Automatización de Tareas Repetitivas

Permiten programar y ejecutar secuencias de operaciones de forma recurrente, como la limpieza de registros antiguos o la generación de informes periódicos.

Aplicación de Reglas de Negocio

Centralizan la lógica de negocio directamente en la base de datos, asegurando que las reglas se apliquen de manera consistente en todas las interacciones con los datos.

Centralización de Lógica

Mueven la lógica que antes residía en la aplicación al servidor de la base de datos, facilitando el mantenimiento y la actualización del código.

Mejora del Rendimiento

Al ejecutar el código directamente en el servidor, se reduce el tráfico de red entre el cliente y el servidor, lo que se traduce en una mayor eficiencia y velocidad.

💡Ejemplo: Un procedimiento almacenado puede validar datos de entrada antes de insertarlos en una tabla, aplicando las reglas de negocio de manera consistente.

Gestión de Permisos

Permisos para Crear

Para crear un procedimiento almacenado, el usuario debe tener el permiso CREATE PROCEDURE:

```
GRANT CREATE PROCEDURE TO 'usuario'@'host';
```

Este permiso se otorga una vez y se aplica a todos los procedimientos que el usuario pueda crear

Permisos para Ejecutar

Para ejecutar un procedimiento almacenado, se requiere el permiso EXECUTE sobre el procedimiento específico:

```
GRANT EXECUTE ON PROCEDURE esquema.nombre TO 'usuario'@'host';
```

```
GRANT EXECUTE ON PROCEDURE base_de_datos.nombre_procedimiento TO 'usuario'@'host';
```

Este permiso debe otorgarse para cada procedimiento específico que se desee ejecutar

Permisos de Acceso a Tablas

Si el procedimiento interactúa con tablas (realizando SELECT, INSERT, UPDATE, DELETE), el usuario que lo ejecuta también debe tener los permisos correspondientes sobre esas tablas.

SELECT INSERT UPDATE DELETE

Los permisos de tabla se verifican en tiempo de ejecución, no cuando se crea el procedimiento

La gestión adecuada de permisos es crucial para la seguridad de la base de datos

Sintaxis de crear

```
DELIMITER //

CREATE PROCEDURE nombre_procedimiento(
  [IN|OUT|INOUT] parametro1 tipo_dato,
  [IN|OUT|INOUT] parametro2 tipo_dato,
  ...
)
BEGIN
  -- Declaraciones opcionales
  DECLARE variable_local tipo_dato;

  -- Sentencias SQL
  -- SET, SELECT, INSERT, UPDATE, etc.
END //

DELIMITER ;
```

```
DELIMITER //


CREATE PROCEDURE sumar(IN a INT, IN b INT, OUT resultado INT)
BEGIN
   DECLARE suma INT;
   SET suma = a + b;
   SET resultado = suma;
END //

DELIMITER ;
```

Es necesario utilizar **DELIMITER** para definir el final del bloque del procedimiento. Dado que en un procedimiento podríasmos hacer múltiples sentencias internas que terminen en “;” y queremos que lea todas las órdenes y las envíe al servidor de una vez en un solo bloque. Por lo que necesita este delimitador especial para ello.

Tipos de Parámetros

IN: Parámetros de Entrada

Valor que se pasa al procedimiento y no puede ser modificado por este.

OUT: Parámetros de Salida

El procedimiento puede asignar un valor a este parámetro, que será devuelto al programa que lo llamó.

INOUT: Parámetros de Entrada y Salida

El valor se pasa al procedimiento, puede ser modificado dentro de este y el nuevo valor se devuelve al programa que lo llamó.

Borrar

Borrar

Para eliminar un procedimiento almacenado que ya no es necesario, se utiliza la instrucción DROP PROCEDURE. Es una buena práctica incluir IF EXISTS para evitar errores.

```
DROP PROCEDURE nombre_procedimiento;

DROP PROCEDURE IF EXISTS nombre_procedimiento;
```

Replace

Usa borrar y crear a la vez

```
DELIMITER //

DROP PROCEDURE IF EXISTS nombre_procedimiento;

CREATE PROCEDURE nombre_procedimiento(
  [IN|OUT|INOUT] parametro1 tipo_dato,
  [IN|OUT|INOUT] parametro2 tipo_dato,
  ...
)
BEGIN
  -- cuerpo del procedimiento
END //

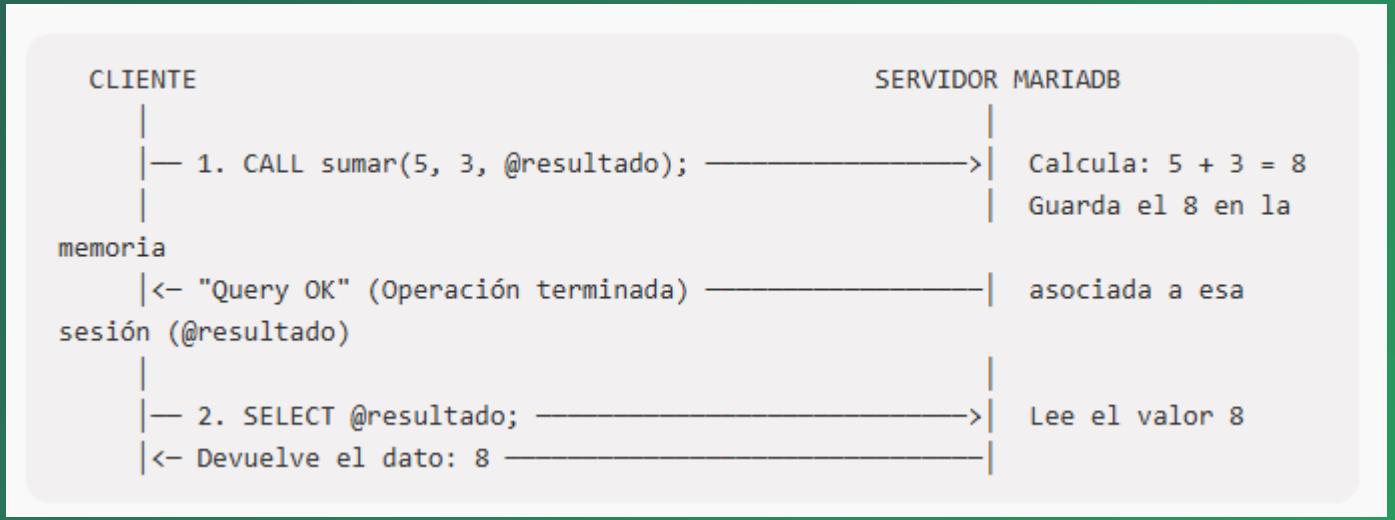
DELIMITER ;
```

Ejecución

Ejecución

La ejecución de un procedimiento almacenado se realiza mediante la instrucción CALL, seguida del nombre del procedimiento y los valores de sus parámetros.

```
CALL sumar(5, 3, @resultado);
SELECT @resultado;
```



Errores Comunes y Soluciones

La gestión de errores es crucial en el desarrollo de procedimientos almacenados. Identificar la causa y aplicar la solución adecuada puede ahorrar tiempo y esfuerzo.

ERROR 1304

Ya existe el procedimiento

Solución:

Usar `DROP PROCEDURE IF EXISTS` antes de crearlo

ERROR 1044

Permiso denegado

Solución:

Conceder permiso `CREATE PROCEDURE` al usuario

ERROR 1064

Error de sintaxis

Solución:

Revisar delimitadores y estructura del bloque

ERROR 1370

No se puede ejecutar

Solución:

Falta permiso `EXECUTE` sobre el procedimiento

Consejo: Siempre verifica los permisos y la sintaxis antes de crear o ejecutar procedimientos almacenados.